

PHASE 1

API, FEEDBACK, SCHEMAS, METRICS, GOVERNANCE & AI SECURITY

Implementation Report and Reproduction Tutorial

Classification: Confidential : For Internal Security Information Only

Report Timestamp

14 June 2026, CEST

Compiled by: Natacha B. and IQA MLOps Project Team

Report Reference: IQA PHASE1 NAT01 NAT10 FINAL VALIDATION 1406

Report Content

This report documents the Phase 1 implementation work performed for the IQA MLOps project on API contracts, feedback lifecycle, Pydantic schemas, AI security metrics and governance controls. It is designed to serve both as a professional implementation report for mentoring review and as a practical tutorial that another team member can replay in a real development environment.

The implementation was developed locally on CubeAI, committed on the branch

Alphabot42/security_threat_model, pushed to the shared GitHub repository, and validated through automated pytest suites. The branch was then replayed on the HP Z420 integration server through a local validation branch that merged origin/main with origin/Alphabot42/security_threat_model. The final technical scope covers IQA1_NAT01 to IQA1_NAT10.

Complementary Phase 1 Deliverables

In complement to the implementation commits, Phase 1 is documented through the following deliverables. The Phase 1 implementation report remains the synthesis document and is updated last after recipe or server validation evidence is added.

Deliverable	Description
IQA_AI_Security_Threat_Model_Report.pdf	Security baseline and threat model used to define the Phase 1 risks, controls and planned security tests.
tests/security/test_api_security_contracts.py	Automated security contract test suite covering schema constraints, feedback lifecycle, reload governance, metrics and audit evidence.
docs/governance/iqa_phase1_governance_matrix.md	Governance matrix mapping Phase 1 controls to AI Act and GDPR aligned principles, without claiming legal compliance.
Phase 1 Recipe Report	Acceptance report listing tested controls, expected results, obtained results, HP Z420 server validation evidence, screenshots and validation conclusions.
PHASE 1 API, FEEDBACK, SCHEMAS, METRICS, GOVERNANCE & AI SECURITY	Final professional report and reproduction tutorial summarizing API, feedback, schemas, metrics, governance and AI security work.

Contents

Report Timestamp.....	1
Report Content.....	1
1. Executive Summary	3
3. Delivered Artifacts	4
4. Implementation Architecture	5
5. Task by Task Implementation Traceability	6
6. Technical Tutorial and Reproduction Guide	6
Environment and branch checkpoint.....	6
Validation command sequence	7
7. API Contracts and Pydantic Schemas	7
Main API import pattern after schema centralization	7
8. Prediction Audit and Feedback Lifecycle	8
Feedback must reference an existing prediction	9
9. Admin Reload Governance	9
Reload acceptance test excerpt.....	10
10. AI Security Metrics and Observability	10
Metrics test excerpt	10
11. Governance Alignment.....	11
12. Security Contract Test Suite	12
13. Test Outputs and Evidence.....	13
Final validation output summary.....	13
14. Git Delivery Evidence	14
15. Known Limits and Next Actions	15
16. Appendices.....	16
Appendix A. End to End Command Sequence	16
Appendix B. Reproduction Workflow Diagram	16
Appendix C. Report References.....	17

1. Executive Summary

Phase 1 transformed the IQA AI security threat model into concrete API controls, feedback safeguards, auditable reload behavior, Prometheus style security metrics and automated security contract tests. The implementation focuses on preventing the most immediate MVP risks: feedback poisoning, invalid feedback, abusive model reload, loss of traceability, unsafe training eligibility and missing governance evidence.

The implementation deliberately remains MVP compatible. Phase 1 delivers executable API controls, schema validation, feedback lifecycle safeguards, reload audit behavior, Prometheus style security metrics, a governance matrix and automated pytest evidence.

Phase 2 must complete production integration work such as persistent storage, live MLflow registry checks, candidate dataset enforcement, ROI warning and ROI fail incident handling, dashboard level monitoring and artifact storage policy. HP Z420 replay is no longer a Phase 2 item because it has been completed and documented in the Phase 1 Recipe Report.

Area	Evidence	Status
Branch	Alphabot42/security_threat_model	Pushed to origin
Main API module	src/iqa/api/main.py	Prediction, feedback, reload and metrics controls
Schemas	src/iqa/api/schemas.py	Centralized Pydantic contracts
Security tests	tests/security/test_api_security_contracts.py	10 dedicated security contract tests
Governance matrix	docs/governance/iqa_phase1_governance_matrix.md	AI Act and GDPR aligned governance mapping
Full validation	pytest full suite	Local validation on CubeAI 85 passed, 1 CUDA warning not blocking Server validation on HP Z420 10 security tests passed, 21 API skeleton tests passed, 92 full suite tests passed after temporary relocation of one ignored local checkpoint artifact

2. System Overview and Phase 1 Scope

IQA is a computer vision quality assistant for industrial Casting parts. In Phase 1, the API does not replace the human quality expert. It provides a controlled support workflow with traceability, prediction identifiers, feedback linkage, model version references, reload controls, metrics and reproducible tests.

Phase 1 is therefore a controlled implementation and validation layer, not the final production platform. All controls described as implemented are backed by code, tests or committed documentation. Items that require persistent services, live registry calls, real monitoring dashboards or final data pipeline enforcement are explicitly treated as Phase 2 follow up work.

Component	Role in Phase 1	Risk controlled
FastAPI gateway	Exposes health, model version, prediction, feedback, replay scenarios, metrics and admin reload routes	Invalid requests, unauthorized reload, missing traceability
Pydantic schemas	Define strict request and response contracts	Uncontrolled input, unexpected fields, missing identifiers
Feedback lifecycle	Links feedback to an existing prediction_id and prevents replay after closure	Feedback poisoning and inconsistent feedback state
Admin reload	Requires an admin token and writes audit events for accepted or refused reloads	Abusive reload and untraceable model switch
Metrics endpoint	Exposes AI security counters through /metrics	Invisible security events and missing monitoring hooks
Security tests	Automates contract, lifecycle, reload and metrics checks	Unverified controls and regression risk

3. Delivered Artifacts

Task	Main files	Delivered value
IQA1_NAT01	docs/security/threat_model_mvp.md, configs/security_controls.yaml, tests/security/README_security_tests.md	Threat model and security backlog established
IQA1_NAT02	src/iqa/api/schemas.py	Pydantic API contracts centralized
IQA1_NAT03	src/iqa/api/main.py	Core API endpoints verified
IQA1_NAT04	src/iqa/api/main.py, tests/test_api_skeleton.py	Prediction response enriched with prediction_id and audit
IQA1_NAT05	src/iqa/api/main.py, src/iqa/api/schemas.py, tests/test_api_skeleton.py	Feedback requires valid open prediction_id
IQA1_NAT06	src/iqa/api/main.py, tests/test_api_skeleton.py	human_sophie display only, oracle_gt train sovereignty
IQA1_NAT07	tests/test_api_skeleton.py	Invalid feedback contracts and conflicts tested
IQA1_NAT08	src/iqa/api/main.py, tests/test_api_skeleton.py	Admin reload audit logging implemented
IQA1_NAT09	src/iqa/api/main.py, tests/test_api_skeleton.py	AI security metrics exposed
IQA1_NAT10	tests/security/test_api_security_contracts.py, docs/governance/iqa_phase1_governance_matrix.md	Security contract suite and governance matrix delivered

4. Implementation Architecture

The Phase 1 implementation follows a layered control architecture. The branch isolates the work, Pydantic schemas enforce API contracts, FastAPI routes execute the controls, audit and metrics create evidence, and pytest validates the behavior before the work is pushed to the shared repository.

Phase 1 Control Stack

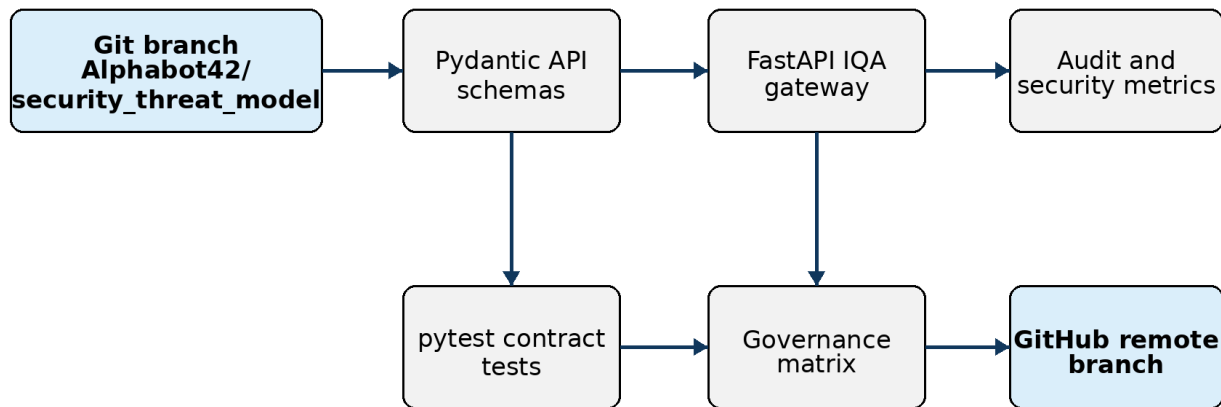


Figure 1. Phase 1 control stack from Git branch to API controls, governance matrix and remote branch.

The same logic is used throughout the implementation: every security requirement must be expressed as either a schema constraint, an API lifecycle guard, an audit event, a metric increment, a pytest test or a governance documentation item.

5. Task by Task Implementation Traceability

Task	Objective	Implementation summary	Commit
NAT01	Define MVP threat model	Created the security baseline covering feedback poisoning, reload abuse, invalid prediction, candidate contamination, scenario collision and secret exposure.	1252196
NAT02	Create Pydantic schemas	Moved API contracts into src/iqa/api/schemas.py with strict schema behavior and reusable request models.	df84142
NAT03	Create core endpoints	Validated /health, /model/version, /replay-scenarios and /metrics baseline behavior.	Covered by API skeleton tests
NAT04	Prediction route with audit	Added prediction_id, model version, audit block and deterministic response structure.	4134042
NAT05	Feedback linked to valid prediction	Added in memory prediction and feedback stores, enforced prediction_id existence and open feedback lifecycle.	0912d1d
NAT06	Human display and GT sovereignty	Allowed human_sophie for display only while keeping oracle_gt sovereign for training eligibility.	43ec323
NAT07	Invalid feedback blocking	Added tests for unknown feedback source, forbidden feedback status, piece mismatch and scenario mismatch.	0e63e1c
NAT08	Admin reload token and audit	Added ADMIN_RELOAD_LOG and audit events for accepted and refused reload attempts.	a0749be
NAT09	AI security metrics	Exposed feedback conflict, AI security incident, unsafe train block, invalid feedback and reload refusal counters.	8cfeba2
NAT10	Security tests and governance matrix	Added dedicated security contract test suite and Phase 1 governance matrix.	718ad77

6. Technical Tutorial and Reproduction Guide

This section allows another team member to reproduce the Phase 1 implementation workflow. It assumes a Linux development environment similar to CubeAI and an already cloned IQA repository.

Environment and branch checkpoint

```
cd /mnt/nvme/mlops/iqa-mlops
source .venv/bin/activate
```

```
git status
git branch --show-current
git branch -vv
python --version
which python
```

Expected result: the working tree must be clean, the active branch must be Alphabot42/security_threat_model, and Python must resolve to the project virtual environment. This prevents accidental commits on main or on another training repository.

Validation command sequence

```
python -m compileall src/iqa/api tests/security
pytest -q tests/security/test_api_security_contracts.py
pytest -q tests/test_api_skeleton.py
pytest -q
```

Expected result: compileall must complete without syntax errors. The local security test suite must pass with 10 tests, the local API skeleton suite must pass with 20 tests, and the local complete test suite must pass with 85 tests. The HP Z420 server replay is documented in the Phase 1 Recipe Report, with 10 security tests passed, 21 API skeleton tests passed and 92 full suite tests passed after temporary relocation of one ignored local checkpoint artifact.

7. API Contracts and Pydantic Schemas

The first implementation step after the threat model was to centralize API request contracts in `src/iqa/api/schemas.py`. The goal was to avoid scattering inline Pydantic classes inside `main.py` and to make API validation reusable by tests, documentation and future service boundaries.

Schema	Core fields	Control objective
PredictRequest	piece_event_id, scenario_id, image_uri	Prediction input must be traceable to a piece event and scenario.
PieceEventPredictRequest	scenario_id, image_uri	The path parameter carries event_id and the body carries scenario and image reference.
FeedbackRequest	prediction_id, piece_event_id, scenario_id, feedback_source	Feedback must be attached to a known prediction and cannot be anonymous.
ReloadModelRequest	scenario_id, stage	Reload must be scoped to an explicit scenario and model stage.
Incident and ModelVersion models	incident metadata, model version metadata	Prepared for Phase 2 incident tracking and model lifecycle evidence.

Main API import pattern after schema centralization

```
from iqa.api.schemas import (
    FeedbackRequest,
    PieceEventPredictRequest,
    PredictRequest,
    ReloadModelRequest,
)
```

Output interpretation: when an unexpected field or missing mandatory field is provided, Pydantic raises `ValidationError`. This is used as a security contract test, not only as input validation.

8. Prediction Audit and Feedback Lifecycle

The prediction route was enriched to return a prediction_id and an audit block. This makes subsequent feedback validation possible and creates a minimal evidence trail directly in the API response.

Prediction and Feedback Governance Flow

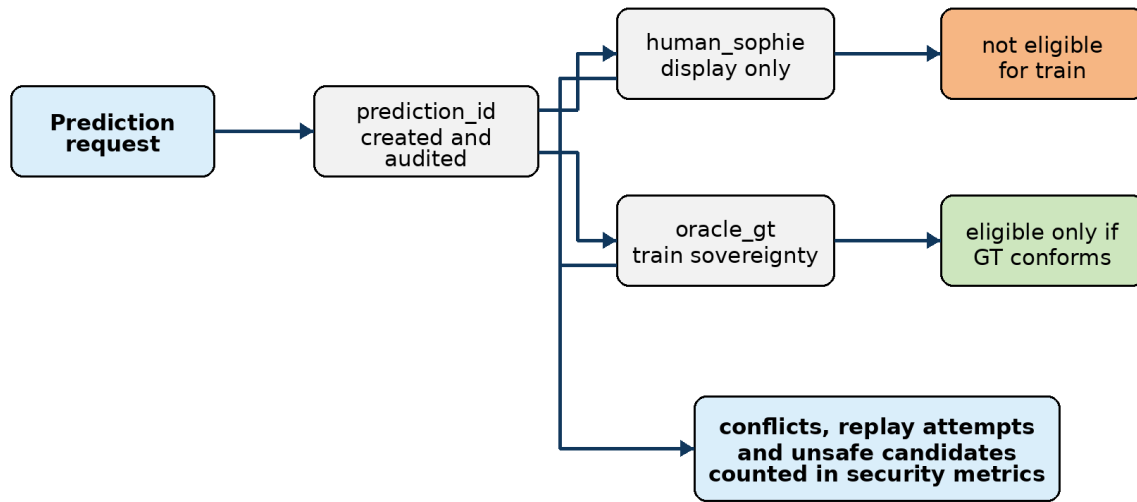


Figure 2. Feedback governance flow separating display decision and training eligibility.

Lifecycle rule	Implementation behavior	Security effect
prediction_id is created	Every prediction receives a unique identifier starting with pred_	Feedback cannot be submitted without being linked to a prediction.
PREDICTION_STORE is updated	MVP in memory store records prediction_id, piece_event_id, scenario_id and feedback_closed	Feedback can be checked against an open prediction.
human_sophie is accepted for display only	feedback_closed remains False and eligible_for_train remains False	Human review can be visible without poisoning training data.
oracle_gt closes the feedback	feedback_closed becomes True	Training eligibility depends on the GT result.
Replayed feedback is refused	HTTP 409 is raised	Duplicate feedback after closure is blocked.

Feedback must reference an existing prediction

```
prediction_response = predict(
    PredictRequest(piece_event_id="piece-security-replay", scenario_id="demo",
        image_uri="s3://bucket/key.jpg")
)
prediction_id = prediction_response["prediction"]["prediction_id"]

feedback(
    FeedbackRequest(
        prediction_id=prediction_id,
        piece_event_id="piece-security-replay",
        scenario_id="demo",
        gt_mask_has_defect=True,
    )
)
```

9. Admin Reload Governance

The reload endpoint changes the active model reference and therefore requires governance. Phase 1 protects the route with IQA_ADMIN_TOKEN and logs every accepted or refused reload attempt. The log is an MVP in memory log. The response keeps MLflow Registry as the declared source of truth, but live MLflow client verification and persistent reload records remain Phase 2 integration work on the target environment.

Admin Reload and Observability Flow

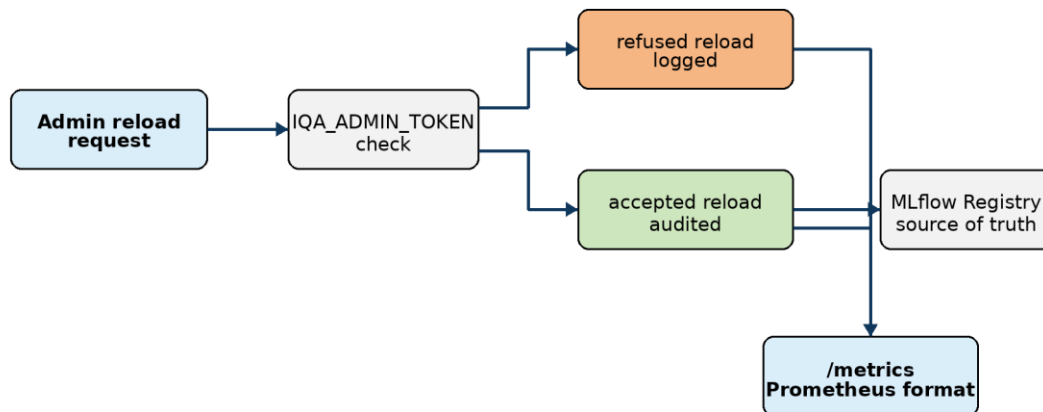


Figure 3. Admin reload control flow and metrics visibility.

Condition	API behavior	Governance meaning
Token missing from environment	HTTP 503	The system is not safely configured for admin reload.
Invalid token provided	HTTP 401	Unauthorized reload is refused.
Valid token provided	accepted True, reload_status accepted	The reload attempt is accepted and audited.
Every decision	ADMIN_RELOAD_LOG receives an audit event	The reload path becomes reviewable and testable.
Source of truth	target is resolved from MLflow registry reference	MinIO artifacts do not decide production state.

Reload acceptance test excerpt

```
monkeypatch.setenv("IQA_ADMIN_TOKEN", "secret")
response = reload_model(
    ReloadModelRequest(scenario_id="demo"),
    x_iqa_admin_token="secret",
)
assert response["audit_logged"] is True
assert response["audit"]["reload_status"] == "accepted"
```

10. AI Security Metrics and Observability

The /metrics endpoint was extended from a simple availability probe to a small Prometheus compatible security surface. The objective is to make security relevant governance events visible for future dashboards, alerts and acceptance evidence.

Metric	Meaning	Use in governance
iqa_feedback_conflict_total	Counts mismatches between prediction_id and piece_event_id or scenario_id	Detects feedback integrity conflicts
iqa_ai_security_incident_total	Counts API detected security events	Aggregates invalid feedback, reload refusal and conflicts
iqa_unsafe_train_blocked_total	Counts feedback or GT outcomes that must not enter normal training	Supports feedback poisoning prevention
iqa_invalid_feedback_total	Counts unknown, replayed or invalid feedback events	Monitors feedback abuse and workflow defects
iqa_reload_refused_total	Counts refused admin reload attempts	Monitors reload abuse or misconfiguration

Metrics test excerpt

```
body = metrics()
assert "iqa_feedback_conflict_total 1" in body
assert "iqa_ai_security_incident_total 1" in body
assert "iqa_unsafe_train_blocked_total 1" in body
```

Output interpretation: a passed metrics test proves that the API not only rejects unsafe behavior, but also exposes the rejection as observable security telemetry.

11. Governance Alignment

The governance layer was made explicit in docs/governance/iqa_phase1_governance_matrix.md. This file does not claim legal compliance. It maps implemented Phase 1 controls to governance principles that are useful for a later compliance discussion, and it separates completed MVP evidence from Phase 2 controls that still require target environment validation or additional pipeline implementation.

Governance theme	Phase 1 alignment	Evidence
AI Act risk management	Threat model and security controls identify feedback poisoning, invalid feedback, unsafe train candidates, reload abuse and traceability loss.	Threat model and NAT01 to NAT10 commits
AI Act data governance	Prediction, feedback and scenario contracts reduce unsafe or untraceable data flow.	schemas.py and feedback tests
AI Act record keeping	Prediction audit, feedback lifecycle and admin reload log provide MVP level records.	API audit response and ADMIN_RELOAD_LOG
AI Act human oversight	Sophie is display priority only and cannot override GT for training eligibility.	human feedback display tests
GDPR purpose limitation	Technical identifiers are used for quality decision traceability and security only.	prediction_id, piece_event_id, scenario_id
GDPR accountability	Controls are documented, tested, committed and reproducible.	Git commits, pytest outputs and reports

External regulatory references used for governance language: Regulation (EU) 2024/1689, Artificial Intelligence Act, and Regulation (EU) 2016/679, General Data Protection Regulation.

12. Security Contract Test Suite

NAT10 delivered a dedicated security test suite in tests/security/test_api_security_contracts.py. This suite is the automated equivalent of a security acceptance checklist. It complements tests/test_api_skeleton.py, which remains the broader API skeleton validation file.

Test	Control area	Expected behavior
test_prediction_contract_rejects_extra_fields	API contract	Unexpected prediction fields raise ValidationError.
test_piece_event_predict_contract_rejects_extra_fields	API contract	Unexpected piece event prediction fields raise ValidationError.
test_feedback_contract_requires_prediction_id	Feedback governance	Feedback without prediction_id raises ValidationError.
test_prediction_security_audit_fields_are_returned	Auditability	Prediction response contains prediction_id and audit block.
test_feedback_security_rejects_replayed_prediction	Feedback lifecycle	Second feedback on a closed prediction raises HTTP 409.
test_feedback_security_conflict_increments_metrics	Observability	Piece mismatch increments conflict and incident metrics.
test_human_feedback_is_display_only_and_not_train_eligible	Human oversight	Human feedback is display only and not train eligible.
test_admin_reload_security_logs_refused_without_configured_token	Reload governance	Missing configured token raises HTTP 503 and logs refusal.
test_admin_reload_security_logs_refused_with_invalid_token	Reload governance	Invalid token raises HTTP 401 and logs refusal.
test_admin_reload_security_accepts_valid_token_and_writes_audit	Reload audit	Valid token accepts reload and writes audit event.

C

13. Test Outputs and Evidence

Validation was first executed locally on CubeAI and then replayed on the HP Z420 integration server. The test evidence proves that the implemented controls are reproducible on the development workstation and on the shared server environment. Detailed HP Z420 evidence, screenshots, expected results and obtained results are documented in the Phase 1 Recipe Report.

```
(iqa-mlops) (base) nat@Cube-AI:/mnt/nvme/mlops/iqa-mlops$ python -m compileall tests/security
pytest -q tests/security/test_api_security_contracts.py
Listing 'tests/security'...
Compiling 'tests/security/test_api_security_contracts.py'...
.....
10 passed in 2.42s
(iqa-mlops) (base) nat@Cube-AI:/mnt/nvme/mlops/iqa-mlops$
```

Figure 4. NAT10 dedicated security test suite with 10 passed.

```
(iqa-mlops) (base) nat@Cube-AI:/mnt/nvme/mlops/iqa-mlops$ python -m compileall src/iqa/api tests/security
pytest -q tests/security/test_api_security_contracts.py
pytest -q tests/test_api_skeleton.py
pytest -q
Listing 'src/iqa/api'...
Listing 'tests/security'...
..... [100%]
10 passed in 2.41s [100%]
..... [100%]
20 passed in 2.43s [100%]
===== warnings summary =====
tests/test_feature_ae_champion_training.py::test_tiny_feature_ae_training_smoke
/mnt/nvme/mlops/iqa-mlops/.venv/lib/python3.12/site-packages/torch/autograd/graph.py:882: UserWarning: CUDA initialization: The NVIDIA driver on your system is too old
(found version 12040). Please update your GPU driver by downloading and installing a new version from the URL: http://www.nvidia.com/Download/index.aspx Alternatively, go
to: https://pytorch.org to install a PyTorch version that has been compiled with your version of the CUDA driver. (Triggered internally at /pytorch/c10/cuda/CUDAFunc
s.cpp:119.)
  return Variable._execution_engine.run_backward( # Calls into the C++ engine to run the backward pass
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
85 passed, 1 warning in 5.25s
(iqa-mlops) (base) nat@Cube-AI:/mnt/nvme/mlops/iqa-mlops$
```

Figure 5. CubeAI local validation with 10 security tests, 20 API skeleton tests and 85 full suite tests passed.

Final validation output summary

```
python -m compileall src/iqa/api tests/security
pytest -q tests/security/test_api_security_contracts.py
pytest -q tests/test_api_skeleton.py
pytest -q
```

CubeAI local validation:
 10 security tests passed
 20 API skeleton tests passed
 85 full suite tests passed, 1 CUDA warning not blocking

HP Z420 server validation:
 10 security tests passed
 21 API skeleton tests passed
 92 full suite tests passed after temporary relocation of one ignored local checkpoint artifact

Output interpretation: the CubeAI warning came from CUDA driver initialization during a tiny Feature AE training smoke test and did not invalidate API, feedback, governance, reload or security contract tests. During HP Z420 validation, the full suite initially failed on a repository hygiene control because an ignored local checkpoint artifact was present under models/. The artifact was not tracked by Git, was temporarily moved outside the repository tree, and the full suite then passed with 92 tests. The checkpoint was restored after validation; details are documented in the Phase 1 Recipe Report.

14. Git Delivery Evidence

The implementation was pushed to GitHub on the remote branch origin/Alphabot42/security_threat_model. After the latest main updates, the branch was also validated on HP Z420 through a local validation branch merging origin/main and origin/Alphabot42/security_threat_model. The branch is intended for review and Pull Request preparation into main.

Commit	Message	Phase task
718ad77	Add IQA security contract tests and governance matrix	NAT10
8cfeba2	Expose IQA AI security metrics	NAT09
a0749be	Add IQA admin reload audit logging	NAT08
0e63e1c	Add IQA invalid feedback contract tests	NAT07
43ec323	Add IQA human feedback display governance	NAT06
0912d1d	Enforce IQA feedback prediction lifecycle	NAT05
4134042	Add IQA prediction audit response	NAT04
df84142	Add IQA API Pydantic schemas	NAT02
1252196	Add IQA MVP AI security threat model	NAT01

```
(iqa-mlops) (base) nat@Cube-AI:/mnt/nvme/mlops/iqa-mlops$ git commit -m "Add IQA security contract tests and governance matrix"
git log --oneline -10
git status
[Alphabot42/security_threat_model 718ad77] Add IQA security contract tests and governance matrix
2 files changed, 279 insertions(+)
create mode 100644 docs/governance/iqa_phase1/governance_matrix.md
create mode 100644 tests/security/test_api_security_contracts.py
718ad77 (HEAD -> Alphabot42/security_threat_model) Add IQA security contract tests and governance matrix
8cfeba2 (origin/Alphabot42/security_threat_model) Expose IQA AI security metrics
a0749be Add IQA admin reload audit logging
0e63e1c Add IQA invalid feedback contract tests
43ec323 Add IQA human feedback display governance
0912d1d Enforce IQA feedback prediction lifecycle
4134042 Add IQA prediction audit response
df84142 Add IQA API Pydantic schemas
1252196 Add IQA MVP AI security threat model
70ca91e (origin/main, origin/HEAD, main) Finalize Phase 1 data contracts and DVC setup
Sur la branche Alphabot42/security_threat_model
Votre branche est en avance sur 'origin/Alphabot42/security_threat_model' de 1 commit.
(utilisez "git push" pour publier vos commits locaux)

rien à valider, la copie de travail est propre
(iqa-mlops) (base) nat@Cube-AI:/mnt/nvme/mlops/iqa-mlops$ git push
Username for 'https://github.com': Alphabot42
Password for 'https://Alphabot42@github.com':
Énumération des objets: 12, fait.
Décompte des objets: 100% (12/12), fait.
Compression par delta en utilisant jusqu'à 4 fils d'exécution
Compression des objets: 100% (8/8), fait.
Écriture des objets: 100% (8/8), 3.98 Kio | 3.98 Mio/s, fait.
Total 8 (delta 3), réutilisés 0 (delta 0), réutilisés du paquet 0 (depuis 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/data-fablab/iqa-mlops.git
8cfeba2..718ad77 Alphabot42/security_threat_model -> Alphabot42/security_threat_model
(iqa-mlops) (base) nat@Cube-AI:/mnt/nvme/mlops/iqa-mlops$
```

Figure 6. Final NAT10 commit and push evidence on the Alphabot42 security branch.

15. Known Limits and Next Actions

The final coherence check below distinguishes what is implemented in Phase 1 from what must remain Phase 2 work. This prevents the report from presenting target architecture items as already completed production controls.

Limit	Phase 1 delivered scope	Phase 2 or later action
MVP in memory stores	PREDICTION_STORE, FEEDBACK_STORE, DISPLAY_FEEDBACK_STORE and ADMIN_RELOAD_LOG are implemented as in memory MVP evidence stores.	Persist prediction, feedback, display feedback and reload audit events to PostgreSQL, JSONL logs or an agreed logging sink during Phase 2 integration.
MLflow production enforcement	Phase 1 returns a registry reference and declares MLflow Registry as the source of truth. It does not yet perform a live MLflow client stage check.	Implement live MLflow client validation, verify prod stage before reload and add integration tests against the target MLflow service.
Candidate dataset governance	Good only, ROI ok only, no defective item and no validation set leakage are defined in the threat model and governance matrix, but the candidate builder is not finalized in NAT10.	Implement candidate_builder controls and tests once the data pipeline contract is stable.
ROI warning and ROI fail	ROI warning and ROI fail are treated as security requirements in the threat model. Phase 1 API tests focus on feedback, reload and metrics controls.	Add ROI warning, ROI fail, incident creation and metric tests when ROI outputs are stable.
Prometheus and Grafana dashboards	Phase 1 exposes Prometheus style counters through /metrics. It does not yet include a deployed dashboard.	Scrape /metrics on the target stack and build Grafana panels for feedback conflicts, invalid feedback, unsafe train blocks and reload refusals.
Z420 validation	HP Z420 replay was completed on a local validation branch merging origin/main and origin/Alphabot42/security_threat_model. Results are documented in the Phase 1 Recipe Report: 10 security tests passed, 21 API skeleton tests passed and 92 full suite tests passed after temporary relocation of one ignored local checkpoint artifact.	Keep the Phase 1 Recipe Report as acceptance evidence. Decide with the team whether ignored local model checkpoints should remain under the repository tree or be moved permanently to an external artifact location.
Legal compliance	The governance matrix is an engineering alignment document. It is not a formal GDPR assessment, DPIA or AI Act classification.	Perform formal legal or compliance review only if required by project governance or deployment scope.

16. Appendices

Appendix A. End to End Command Sequence

```
cd /mnt/nvme/mlops/iqa-mlops
source .venv/bin/activate
```

```
git status
git branch --show-current
git branch -vv
```

```
python -m compileall src/iqa/api tests/security
pytest -q tests/security/test_api_security_contracts.py
pytest -q tests/test_api_skeleton.py
pytest -q
```

```
git add tests/security/test_api_security_contracts.py
docs/governance/iqa_phase1_governance_matrix.md
git commit -m "Add IQA security contract tests and governance matrix"
git push
```

HP Z420 acceptance replay is documented in the Phase 1 Recipe Report.

Appendix B. Reproduction Workflow Diagram

Reproduction Workflow

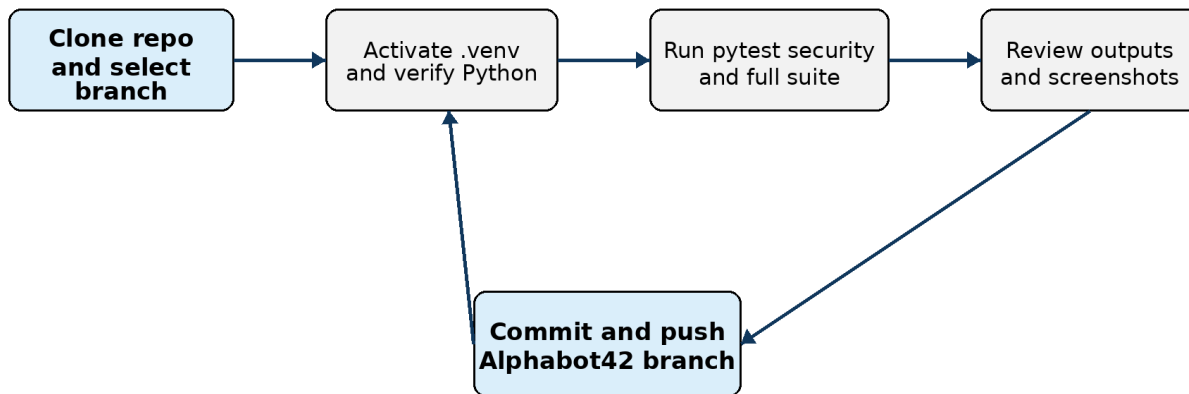


Figure 7. Reproduction workflow for a team member replaying the work in a real environment.

Appendix C. Report References

Reference	Use in report
IQA threat model	IQA AI Security Threat Model Report, latest uploaded Word template
Repository branch	origin/Alphabot42/security_threat_model
AI governance reference	Regulation (EU) 2024/1689, Artificial Intelligence Act
Data governance reference	Regulation (EU) 2016/679, General Data Protection Regulation
Test framework	pytest outputs captured on CubeAI and HP Z420 server
Phase 1 Recipe Report	HP Z420 acceptance evidence, validation screenshots, checkpoint artifact diagnostic and final test conclusions